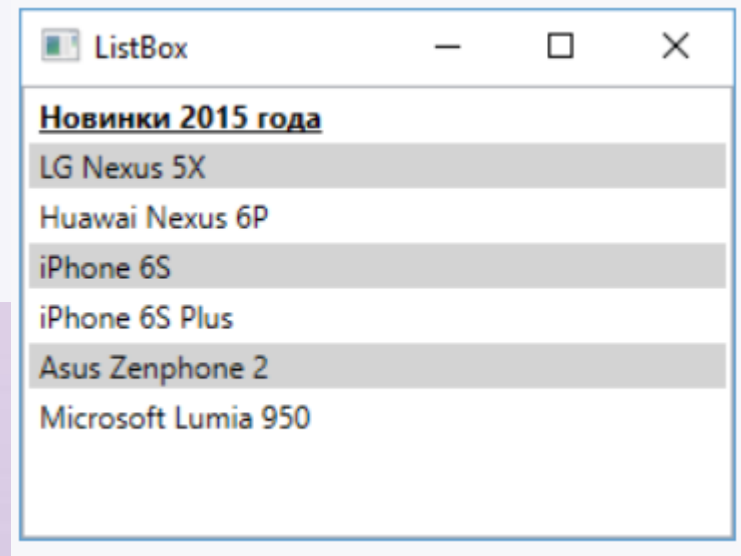


Список ListBox

ListBox

Представляет собой обычный список. Содержит коллекцию элементов `ListBoxItem`, которые являются типичными элементами управления содержимым. Также `ListBox` может содержать любые другие элементы, например:

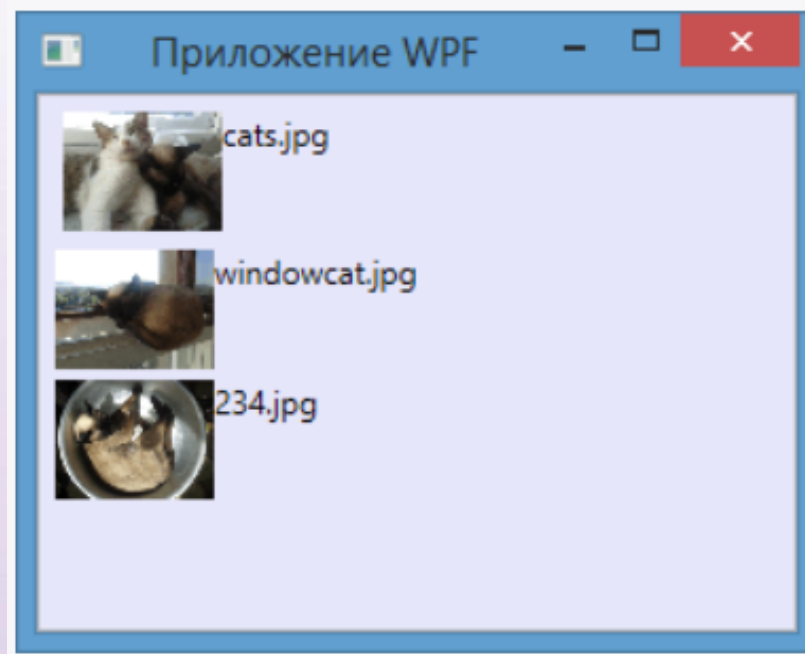
```
<ListBox Name="phonesList">
    <TextBlock FontWeight="Bold" TextDecorations="Underline" Text="Новинки 2015 года" />
    <ListBoxItem Background="LightGray">LG Nexus 5X</ListBoxItem>
    <ListBoxItem>Huawei Nexus 6P</ListBoxItem>
    <ListBoxItem Background="LightGray">iPhone 6S</ListBoxItem>
    <ListBoxItem>iPhone 6S Plus</ListBoxItem>
    <ListBoxItem Background="LightGray">Asus Zenphone 2</ListBoxItem>
    <ListBoxItem>Microsoft Lumia 950</ListBoxItem>
</ListBox>
```



Все эти элементы будут находиться в коллекции `phonesList.Items` и, таким образом, по счетчику можно к ним обращаться, например, `phonesList.Items[0]` - первый элемент `ListBox`, который в данном случае представляет `TextBlock`. Также мы можем установить элемент: `phonesList.Items[2]="LG G 4";`

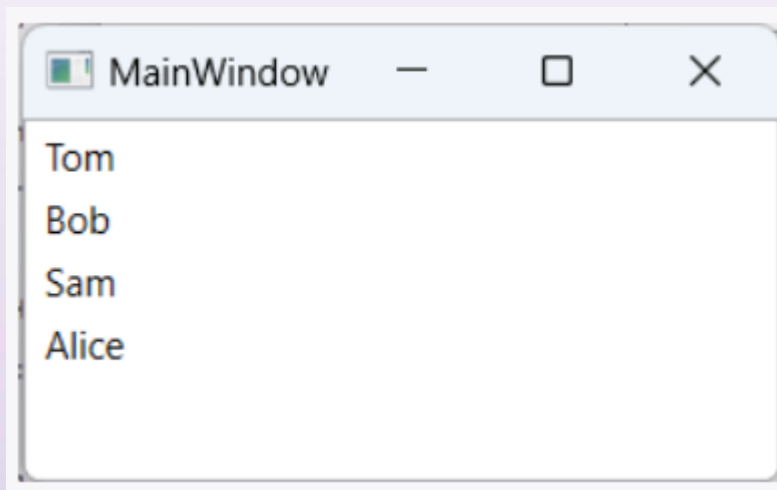
Компонент `ListBoxItem` представляет элемент управления содержимым, поэтому также мы можем задавать через его свойство `Content` более сложные композиции элементов.

```
<ListBox Name="Photos" Background="Lavender">
  <ListBoxItem Margin="3">
    <StackPanel Orientation="Horizontal">
      <Image Source="cats.jpg" Width="60" />
      <TextBlock>cats.jpg</TextBlock>
    </StackPanel>
  </ListBoxItem>
  <StackPanel Orientation="Horizontal">
    <Image Source="windowcat.jpg" Width="60" />
    <TextBlock>windowcat.jpg</TextBlock>
  </StackPanel>
  <StackPanel Orientation="Horizontal">
    <Image Source="234.jpg" Width="60" />
    <TextBlock>234.jpg</TextBlock>
  </StackPanel>
</ListBox>
```



Простейший элемент-список - **ListBox**

```
<ListBox x:Name="usersList">  
  <sys:String>Tom</sys:String>  
  <sys:String>Bob</sys:String>  
  <sys:String>Sam</sys:String>  
  <sys:String>Alice</sys:String>  
</ListBox>
```



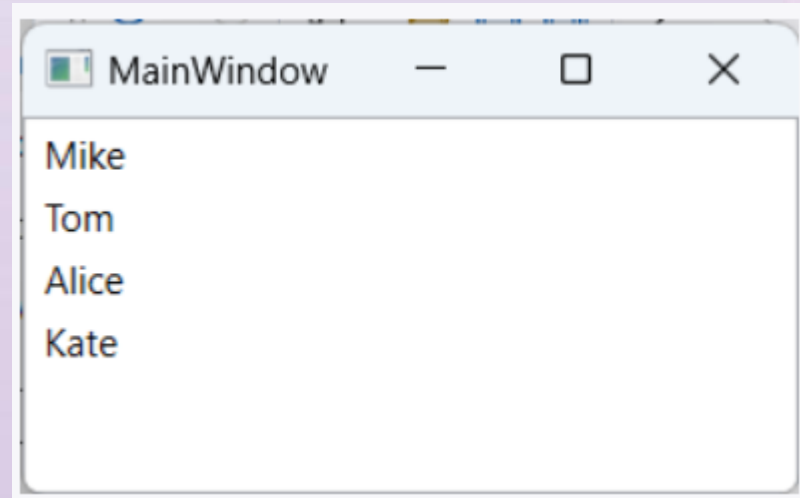
Управление списком

Коллекция объектов внутри элемента-списка доступна в виде свойства **Items**. Для управления элементами из этой коллекции мы можем использовать следующие методы:

- **Add(object item)**: добавление элемента
- **Clear()**: полная очистка коллекции
- **Insert(int index, object item)**: вставка элемента по определенному индексу в коллекции
- **Remove(object item)**: удаление элемента
- **RemoveAt(int index)**: удаление элемента по индексу

А свойство **Count** позволяет узнать, сколько элементов в коллекции.

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        usersList.Items.Remove("Sam"); // удаляем элемент "Sam"
        usersList.Items.RemoveAt(1); // удаляем второй элемент
        usersList.Items.Add("Kate"); // Добавляем элемент "Kate"
        usersList.Items.Insert(0, "Mike"); // Вставляем элемент "Mike" на первое место в списке
    }
}
```



Отображение сложных данных

Списковые элементы могут отображать не только простые строки или числа, но и сложные данные. Например, создадим в коде новый класс Person:

```
Ссылка: 2
public partial class MainWindow ...

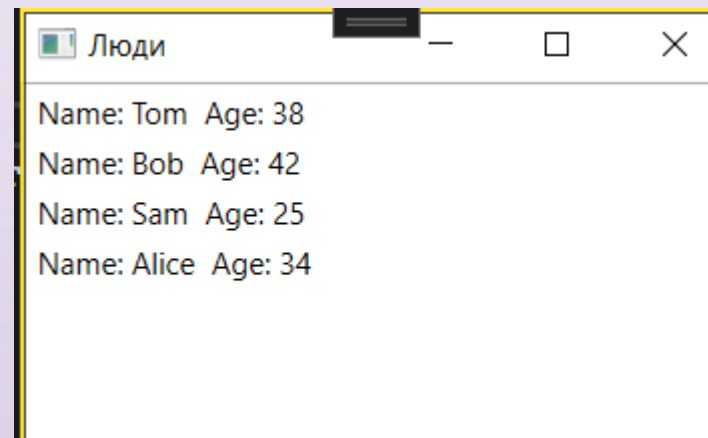
Ссылка: 0
public class Person
{
    Ссылка: 1
    public string Name { get; set; } = "";
    Ссылка: 1
    public int Age { get; set; }

    Ссылка: 0
    public override string ToString()
    {
        return $"Name: {Name} Age: {Age}";
    }
}
```


Должно быть подключено `xmlns:local="clr-namespace:WpfAppName"`

После записи `local:Person` нужно пересобирать проект.

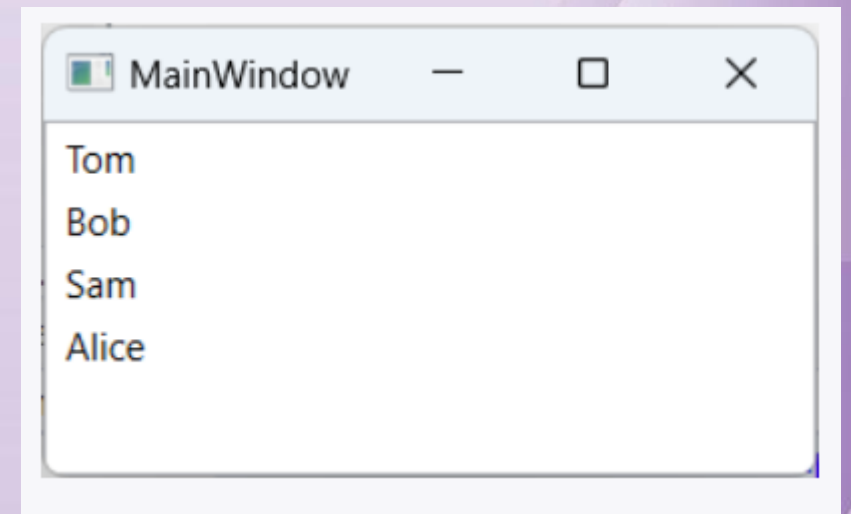
```
<Grid>
  <ListBox x:Name="usersList">
    <local:Person Name="Tom" Age="38" />
    <local:Person Name="Bob" Age="42" />
    <local:Person Name="Sam" Age="25" />
    <local:Person Name="Alice" Age="34" />
  </ListBox>
</Grid>
```



DisplayMemberPath

По умолчанию списковые элементы выводят то, что возвращает метод ToString() объекта. Именно для этой демонстрации и реализован данный метод в классе Person выше. Но также с помощью свойства DisplayMemberPath можно установить свойство объекта, значение которого будет отображаться в списке. Например, отобразим только имя

```
<ListBox x:Name="usersList" DisplayMemberPath="Name">  
    <local:Person Name="Tom" Age="38" />  
    <local:Person Name="Bob" Age="42" />  
    <local:Person Name="Sam" Age="25" />  
    <local:Person Name="Alice" Age="34" />  
</ListBox>
```



ItemsSource

Нам необязательно вручную заполнять значения элемента управления списком, так как мы можем установить свойство **ItemsSource**, задав в качестве параметра коллекцию, из которой будет формироваться элемент управления списком. Например, в коде xaml-разметки определим пустой список.

```
<ListBox x:Name="usersList" />
```

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        usersList.ItemsSource = new List<Person>()
        {
            new Person{Name="Tom", Age=38},
            new Person {Name="Bob", Age=42},
            new Person{Name="Sam", Age=25}
        };
        // устанавливаем отображаемое свойство
        usersList.DisplayMemberPath = "Name";
    }
}
```

ItemsSource - изменение

Но стоит учитывать, что если мы наполняем элемент управления списком через свойство ItemsSource, то мы не сможем использовать выше рассмотренные методы Add/Remove/RemoveAt/Insert/Clear для добавления/удаления элементов списка. В этом случае для управления элементами непосредственно к самому источнику данных.

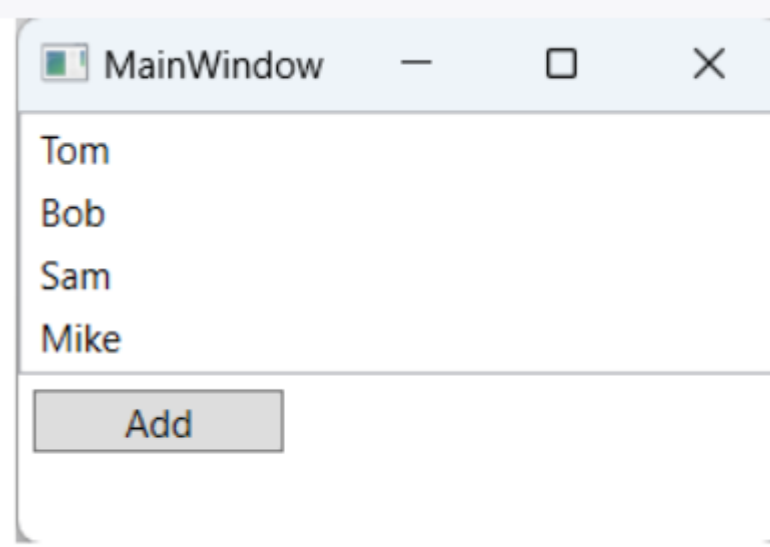
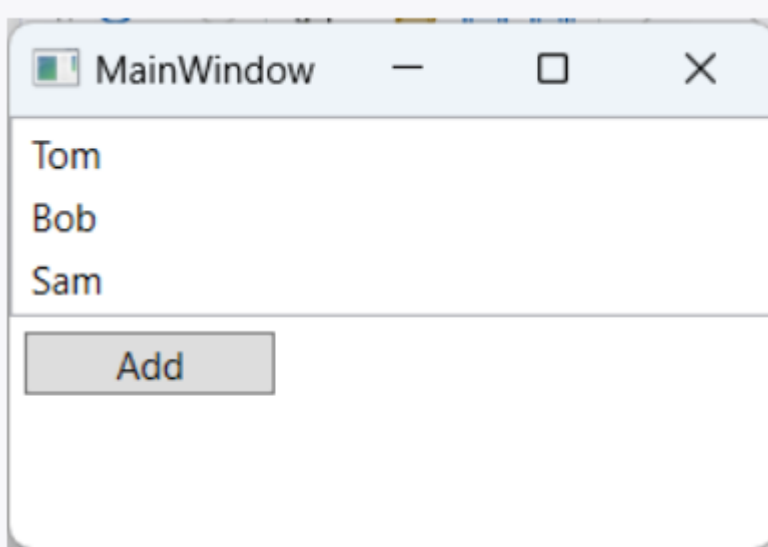
```
<StackPanel>
    <ListBox x:Name="usersList" DisplayMemberPath="Name" />
    <Button Content="Add" Click="Button_Click" Margin="5" Width="80" HorizontalAlignment="Left" />
</StackPanel>
```

```
public partial class MainWindow : Window
{
    List<Person> people = new List<Person>()
    {
        new Person{Name="Tom", Age=38},
        new Person {Name="Bob", Age=42},
        new Person{Name="Sam", Age=25}
    };
    public MainWindow()
    {
        InitializeComponent();
        usersList.ItemsSource = people;
    }
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        people.Add(new Person { Name = "Mike", Age = 29 });
        usersList.Items.Refresh();
    }
}
```

Здесь в качестве источника данных выступает список `people`, и из этого списка `ListBox` будет брать данные.

В обработчике нажатия кнопки добавляем в список `people` один объект `Person` для теста. Обратите внимание, что добавляем именно в стандартный список `List<Person>`, а не в `ListBox`. `ListBox` в данном случае просто отображает данные. Но чтобы `ListBox` обновил свое визуальное представление, после изменения списка `people` обновляем данные в `ListBox`:

```
usersList.Items.Refresh();
```



Выбор элементов

Все элементы управления списками поддерживают выделение входящих элементов. Выделенный элемент(ы) можно получить с помощью свойств **SelectedItem**(**SelectedItems**), а получить индекс выделенного элемента - с помощью свойства **SelectedIndex**. Свойство **SelectedValue** позволяет получить значение выделенного элемента.

При выделении элемента в списке генерируется событие **SelectionChanged**, которое мы можем обработать.


```
<ListBox x:Name="usersList" DisplayMemberPath="Name" SelectionChanged="usersList_SelectionChanged">
    <local:Person Name="Tom" Age="38" />
    <local:Person Name="Bob" Age="42" />
    <local:Person Name="Sam" Age="25" />
    <local:Person Name="Alice" Age="34" />
</ListBox>
```

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
    private void usersList_SelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        if (usersList.SelectedItem is Person person)
        {
            MessageBox.Show(person.Name);
        }
    }
}
```

Задание - задачник

Создайте приложение для ведения задач. На окне будет отображён заголовок приложения, список с задачами и три кнопки: информация о программе, добавление задачи, удаление задачи.

Класс «Задача» с полями: название, описание, количество дней на выполнение.

Кнопка «*Информация о программе*» отображает MessageBox с информацией.

Перед кнопкой «*добавить задачу*» на окне лежат три поля ввода для заполнения информации. После нажатия кнопки у пользователя спрашивают «Ты хочешь добавить задачу {Название} {Описание} на {кол. дней} дня?». После чего добавляется или не добавляется задача.

Кнопка «*удалить*» удаляет текущую выбранную задачу в списке(SelectedItem). Прежде чем удалять задачу требуется переспросить пользователя, вдруг он промахнулся ☹.